

Miary perplexity a jakość agentów LLM

Raport projektu — Warsztaty AI

Klaudia Chwistek

Maj 2026

1 Wprowadzenie

Perplexity (PPL) powstało jako miara jakości modelu językowego - mówi, jak bardzo model jest zaskoczony obserwowanym tekstem. Z czasem zaczęto je jednak traktować szerzej: jako sygnał o tym, czy model dobrze radzi sobie z konkretnym zadaniem. Pomysł jest zachęcający zwłaszcza w agentach LLM, gdzie taki sygnał mógłby sterować decyzjami - kazać agentowi wstrzymać się z odpowiedzią, poprosić o pomoc albo przeplanować działanie, gdy jego pewność spada.

Literatura nie jest tu jednak zgodna. Część prac pokazuje, że niższe PPL wiąże się z lepszymi wynikami, inne - że PPL bywa zawodne i podatne na artefakty długości czy interpunkcji. Większość tych badań dotyczy przy tym pojedynczych, statycznych promptów. Tymczasem agent nie produkuje jednej odpowiedzi, lecz całą trajektorię - naprzemienne kroki rozumowania, wywołań narzędzi i obserwacji - i nie jest oczywiste, czy wnioski z analizy pojedynczych promptów dają się na taki przypadek przenieść.

W raporcie opisano badania tej metryki. Sprawdzane jest, czy perplexity i pokrewne sygnały rozkładowe (entropia tokenowa, top-1 margin) przewidują poprawność odpowiedzi małego modelu LLM na zadaniach matematycznych z GSM8K, w dwóch ustawieniach: czystego chain-of-thought oraz agenta ReAct z kalkulatorem. Osobno badane jest, w którym miejscu trajektorii agenta sygnał ten jest najsilniejszy.

Z eksperymentów wyłaniają się trzy obserwacje. Metryki korzystające z całego rozkładu prawdopodobieństw (entropia, margin) konsekwentnie przewidują poprawność lepiej niż samo perplexity. Wewnątrz agenta sygnał nie jest rozłożony równomiernie - najsilniej koreluje z poprawnością pewność na segmentach formułujących wywołania narzędzia, a praktycznie zanika na finalnej odpowiedzi. Wreszcie, choć pewność porządkuje pytania sensownie, jako bezwzględna miara jest źle wykalibrowana: model jest systematycznie zbyt pewny siebie. Ta ostatnia obserwacja potwierdza sprzeczne stanowiska literatury - PPL jest użytecznym sygnałem porządkowym, ale zawodnym sygnałem ilościowym.

Dalsza część pracy ma następującą strukturę. Sekcja 2 omawia tło teoretyczne i powiązane prace. Sekcja 3 opisuje zbiór danych, modele, sposób skoringu i badane metryki. Sekcja 4 przedstawia wyniki korelacyjne w trzech ustawieniach, a sekcja 5 - analizę kalibracji. Sekcja 6 zestawia obserwacje z literaturą, a sekcja 7 omawia ograniczenia.

2 Tło teoretyczne

2.1 Perplexity jako miara modelu językowego

Perplexity (PPL) to średnia geometryczna odwrotności prawdopodobieństw, jakie model przypisuje kolejnym tokenom danej sekwencji. Dla sekwencji $x_1, \dots, x_N$ pod modelem p_θ :

$$\text{PPL}(x_{1:N}) = \exp\left(-\frac{1}{N} \sum_{t=1}^N \log p_\theta(x_t \mid x_{<t})\right). \quad (1)$$

Niższa wartość oznacza, że model był mniej zaskoczony tym, co zobaczył. Klasycznie PPL używa się jako miary jakości samego modelu językowego - im niższe na zbiorze testowym, tym lepiej model przewiduje tekst. W tej pracy interesuje jednak inne zastosowanie: PPL liczone na własnej odpowiedzi modelu, jako sygnał mówiący o tym, czy ta odpowiedź jest poprawna - taki, żeby agent mógł np. odroczyć odpowiedź lub poprosić o pomoc, gdy jego pewność spada.

2.2 Argumenty za perplexity

Gonen i in. [1] pokazali, że perplexity samego promptu jest dobrym predyktorem skuteczności modelu - prompty o niższym PPL dają wyższą trafność odpowiedzi. Interpretacja autorów: prompty, które „brzmia naturalniej” dla modelu (mniejsze zaskoczenie na jego tokenach), są dla niego łatwiejsze i prowadzą do lepszych wyników. Sam pomiar PPL działa tu jako tania heurystyka wyboru lepszego promptu - bez konieczności faktycznego uruchamiania zadania i sprawdzania wyniku.

2.3 Argumenty przeciw perplexity

Wang i in. [2] pokazują drugą stronę medalu: PPL bywa zawodne jako miara jakości tekstu. Autorzy zwracają uwagę na trzy pułapki. Po pierwsze, krótsze teksty dostają wyższe PPL niezależnie od jakości (*length bias*). Po drugie, powtarzające się fragmenty sztucznie je zaniżają (*repetition sensitivity*). Po trzecie, drobne różnice w interpunkcji potrafią wprowadzać dużą zmienność (*punctuation dependency*). Zestawienie tych dwóch prac otwiera pytanie istotne dla przeprowadzonych eksperymentów: czy ten obraz zmienia się, gdy zamiast pojedynczego promptu analizowane są wieloetapowe trajektorie agentów?

2.4 Perplexity w długim kontekście

Fang i in. [3] pokazują problem klasycznego PPL na zadaniach z długim kontekstem. Standardowe PPL uśrednia logarytmy prawdopodobieństw po wszystkich tokenach z taką samą wagą, przez co całość zdominowana jest przez tokeny lokalnie przewidywalne - spójniki, końcówki fleksyjne, znaki interpunkcyjne - a wkład tokenów, które naprawdę zależą od dalekiego kontekstu, ginie w średniej. Zaproponowana przez autorów metryka LongPPL waży tokeny zgodnie z tym, jak bardzo zależą od dalekiego kontekstu; na benchmarkach długiego kontekstu osiąga ona korelację Pearsona $r \approx -0,96$ z faktyczną wydajnością modeli - podczas gdy zwykle PPL daje na tych samych danych wartość bliską zeru.

Ten sam mechanizm może działać też w przypadku trajektorii agentów. Trajektorie agenta ReAct - złożone z naprzemiennych bloków **Thought**, **Action**, **Observation** i **Answer** - są długie i pełne tokenów strukturalnych (samych etykiet segmentów, znaków interpunkcyjnych), na których model jest bardzo pewny niezależnie od jakości rozwiązania. Takie tokeny mogą zdominować jedną wartość PPL liczoną na całej trajektorii i wyłumić sygnał z miejsc, w których model rzeczywiście „myśli” - na przykład formułuje wywołanie kalkulatora. Stąd w tej pracy oprócz PPL na całej trajektorii liczone jest też PPL osobno na segmentach **Action** i **Answer** (sprawdzone w sekcji 4.2). Szerszy przegląd metryk ewaluacji agentów - w którym PPL pojawia się obok trafności wywołań narzędzi, jakości planu, metryk trajektorii i bezpieczeństwa - można znaleźć w pracy Mohammadi i in. [4].

2.5 Niepewność jako sygnał sterujący w agentach

Trzy nowsze artykuły idą krok dalej: zamiast tylko mierzyć pewność modelu, wpinają ją wprost w pętlę decyzyjną agenta. ReDAct [5] wprowadza próg pewności, poniżej którego agent odracza odpowiedź zamiast ryzykować błąd. *Every Response Counts* [6] zbiera niepewność wielu agentów pracujących nad tym samym zadaniem i - korzystając z dekompozycji tensorowej - próbuje wyłowić, kiedy modele systematycznie się ze sobą nie zgadzają. *Towards Agents That Know When They Don't Know* [7] traktuje niską pewność jako sygnał, że trzeba się zatrzymać i sprawdzić aktualny krok rozumowania, zanim agent pójdzie dalej.

Wszystkie trzy prace milcząco zakładają, że pewność liczona z rozkładu tokenów rzeczywiście mówi coś o jakości tego, co model generuje. W tej pracy sprawdzane jest, na ile to założenie się utrzymuje na prostym agencie ReAct rozwiązującym zadania matematyczne: jak silna jest korelacja PPL i entropii z poprawnością i czy ma znaczenie, na której części trajektorii się ją mierzy.

3 Metodologia

3.1 Pytanie badawcze

W tej pracy sprawdzane jest, czy perplexity i pokrewne sygnały liczone z rozkładu prawdopodobieństw tokenów - średnia entropia tokenowa, top-1 margin, top-1 mass - przewidują poprawność odpowiedzi LLM na konkretne pytanie. Pytanie to stawiane jest w dwóch ustawieniach: w czystym chain-of-thought (CoT) oraz w agencie ReAct z pojedynczym narzędziem.

Drugie, pomocnicze pytanie dotyczy tego, gdzie wewnątrz trajektorii agenta najlepiej mierzyć tę pewność: czy mocniejszy sygnał płynie z segmentów *action* (wywołań narzędzia), *thought* (rozumowania) czy *answer* (finalnej odpowiedzi).

3.2 Zbiór danych

Eksperymenty przeprowadzono na zbiorze GSM8K [8] - 8500 zadań tekstowych z matematyki szkolnej (7473 trenujących i 1319 testowych), z których każde wymaga od dwóch do ośmiu kroków rozumowania. Każde zadanie kończy się jednoznaczną odpowiedzią liczbową poprzedzoną sekwencją ####, dzięki czemu ocena poprawności sprowadza się do porównania dwóch liczb - bez potrzeby angażowania drugiego modelu jako sędziego. Przykład z części testowej:

Natalia sprzedała spinacze 48 swoim przyjaciółkom w kwietniu, a w maju sprzedała ich o połowę mniej. Ile spinaczy łącznie sprzedała w kwietniu i maju?

Rozwiązanie: $48/2 = 24$ spinaczy w maju; $48 + 24 = 72$ spinaczy łącznie. #### 72

GSM8K został użyty w tym projekcie z trzech powodów. Po pierwsze, ocena jest zerojedynkowa i obiektywna - liczba albo się zgadza, albo nie. Po drugie, zadania są krótkie i mieszczą się w oknie kontekstowym małych modeli (0,5-1,5B parametrów). Po trzecie, modele tej skali osiągają na GSM8K dokładność rzędu 20-60%, czyli ani nie zgadują na ślepo, ani nie rozwiązują wszystkiego - co jest pożądane przy badaniu korelacji pewności z poprawnością. GSM8K jest też standardowym benchmarkiem dla prac o rozumowaniu krok po kroku od [9].

Z części testowej losowano 50 zadań (faza pilotażowa) lub 60 zadań (eksperymenty główne) przy ustalonych ziarnach. Modele nie były dotrenowywane - wszystkie wyniki pochodzą z czystej ewaluacji.

3.3 Modele

W eksperymencie użyto dwóch modeli z tej samej rodziny, różniących się tylko skalą: Qwen2.5-0.5B-Instruct (w fazie pilotażowej) oraz Qwen2.5-1.5B-

Instruct (w eksperymentach głównych). Pozwala to sprawdzić, czy obserwacje z fazy pilotażowej utrzymują się przy przejściu na większy model.

3.4 Dekodowanie i skoring

We wszystkich eksperymentach stosowane jest dekodowanie zachłanne (*greedy decoding*) - na każdym kroku wybierany jest token o najwyższym prawdopodobieństwie. Eliminuje to szum samplingu i sprawia, że wartości PPL między pytaniami można porównywać. Ma to swoją cenę: ponieważ model zawsze wybiera swój top-1 token, jego prawdopodobieństwa są systematycznie wysokie, a wartości PPL skupiają się blisko 1,0. Jest to jedno z ograniczeń, do których raport wraca w sekcji 7.

Skoring działa w jednym przejściu modelu przez tekst złożony z promptu i wygenerowanej kontynuacji. Tokeny promptu są maskowane, więc do metryk wliczają się tylko te tokeny, które model sam wygenerował - mierzona jest jego pewność co do własnego rozumowania, a nie znajomość samego polecenia. Implementacja znajduje się w pliku `experiments/perplexity.py` (funkcja `score_continuation`).

3.5 Dwa warianty: CoT i agent ReAct

Pierwszy wariant to czysty chain-of-thought: model dostaje prompt zachęcający do myślenia krok po kroku i generuje całą odpowiedź w jednym przebiegu, bez narzędzi.

Drugi wariant to pętla ReAct [10], w której model na przemian produkuje segmenty oznaczone **Thought:**, **Action:**, **Observation:** i **Answer:**. Jedynym dostępnym narzędziem jest kalkulator, a liczba wywołań w jednym pytaniu ograniczona jest do pięciu.

Te same 60 pytań i ten sam model (Qwen2.5-1.5B) w obu wariantach pozwalają czysto porównać, czy włożenie modelu w pętlę agentową zmienia samą relację między PPL a poprawnością - nawet jeśli dokładność spada lub rośnie.

3.6 Metryki

Rozkład prawdopodobieństw tokenów sprowadzany jest do czterech skalarów. Pierwszy z nich, *perplexity*, dane jest równaniem (1) i mówi, jak bardzo średnio model był zaskoczony własnymi tokenami. *Średnia entropia tokenowa* - $\sum_v p(v) \log p(v)$, uśredniona po wszystkich pozycjach sekwencji, korzysta z całego rozkładu, a nie tylko z tokena, który został wybrany - dzięki temu reaguje także wtedy, gdy top-1 token jest pewny, ale za nim stoi „gęsta” alternatywa. *Top-1 margin* to różnica $p(\text{top-1}) - p(\text{top-2})$, czyli to, jak bardzo wybrany token wygrał z najbliższym konkurentem. *Top-1 mass* to po prostu $p(\text{top-1})$.

Do oceny kalibracji wykorzystano *Expected Calibration Error* (ECE), liczone na binach o równej liczbie obserwacji ($n_{\text{bins}} = 10$). Pewność modelu definiowana jest tu jako $1/\text{PPL}$. Niższe ECE oznacza lepszą zgodność tego, jak pewny był model, z tym, jak często rzeczywiście miał rację.

4 Wyniki

Wszystkie wyniki pochodzą z jednego przebiegu kodu z katalogu `experiments/`, na ustalonych ziarnach losowości. Surowe CSV z wynikami zapisane są w katalogu `overnight_results/` i można je odtworzyć z dołączonego notebooka `main_results_report.ipynb`.

4.1 Faza pilotażowa: Qwen-0,5B, czysty CoT, $n = 150$

Na 150 zadaniach (trzy ziarna po 50) model Qwen2.5-0,5B-Instruct osiąga dokładność **44,7%** - mieszczącą się w przedziale, który dobrze nadaje się do badania korelacji pewności z poprawnością.

Już na tej skali widać pierwszy systematyczny wzorec: średnia entropia tokenowa jest lepszym predyktorem poprawności niż samo perplexity ($r = -0,317$, $p < 0,001$ vs $r = -0,236$, $p = 0,004$). Innymi słowy, sygnał z całego rozkładu prawdopodobieństw niesie więcej informacji niż średnia liczona tylko po tokenach, które model faktycznie wybrał. Ten sam wzorec powtarza się w każdym kolejnym ustawieniu.

4.2 Agent ReAct z kalkulatorem: Qwen-1,5B, $n = 60$

Na 60 zadaniach (dwa ziarna po 30) Qwen2.5-1,5B-Instruct w pętli ReAct z pojedynczym narzędziem (kalkulator) osiąga dokładność **28,3%**. Spośród 60 trajektorii 12 (20%) zostaje obciętych - model albo przekracza limit pięciu wywołań kalkulatora, albo łamie format dialogowy (np. zapomina o etykiecie `Action:`).

Najciekawsza obserwacja w agencji dotyczy tego, *gdzie* warto mierzyć pewność. Metryki liczone tylko na segmentach `Action:` - czyli tam, gdzie model formułuje wywołanie kalkulatora - dają najsilniejszą korelację z poprawnością: `action_ppl_mean` osiąga $r = -0,436$, $p = 0,002$, $n = 48$, z bootstrapowym 95% CI $[-0,60, -0,24]$ (5000 resamplowań). Te same metryki liczone tylko na finalnej odpowiedzi (`Answer:`) praktycznie nie niosą żadnego sygnału ($r = -0,099$, $p = 0,50$, CI $[-0,32, +0,21]$ - przedział ufności pokrywa zero).

Intuicja stojąca za tym wynikiem jest dość prosta. W momencie, w którym model generuje finalną odpowiedź, wynik liczbowy znajduje się już w jego kontekście - widoczny jako rezultat ostatniego wywołania kalkulatora. Model w zasadzie kopiuje go z kontekstu, więc jest bardzo pewny niezależnie

od tego, czy całe rozumowanie było prawidłowe. Pewność modelu w trakcie *formułowania* wywołań - gdy musi zdecydować, co policzyć - rozróżnia poprawne i błędne odpowiedzi. Pewność na samej *konkluzji* - już nie.

Sam rozkład pewności też różni się między typami segmentów. Segmenty **Answer**: są skupione najmocniej (średnie PPL= 1,26, top-1 margin = 0,80), segmenty **Action**: - pośrednio (PPL= 1,46, margin = 0,64), segmenty **Thought**: - najmniej (PPL= 1,59, margin = 0,61). Ten porządek utrzymuje się w obu ziarnach.

4.3 Kontrola czystego CoT: Qwen-1,5B, $n = 60$

Żeby zobaczyć, jak na te same pytania reaguje model bez pętli agentowej, te same 60 zadań przepuszczono jeszcze raz przez Qwen2.5-1,5B-Instruct, tym razem w czystym CoT. Dokładność rośnie do **60,0%** (wobec 28,3% w wariancie agentowym), a najsilniejszy sygnał korelacyjny w całym projekcie pojawia się właśnie tutaj: średnia entropia tokenowa łańcucha rozumowania osiąga $r = -0,490$, $p < 0,001$, $n = 60$. Samo perplexity też koreluje z poprawnością, ale słabiej: $r = -0,283$, $p = 0,028$. Włożenie modelu w pętlę ReAct nie likwiduje więc sygnału PPL/entropii - przenosi go z całej kontynuacji na segmenty wywołań narzędzia.

4.4 Porównanie zbiorcze

Tabela 1 zestawia korelacje Pearsona między metrykami a poprawnością w trzech głównych ustawieniach.

Ustawienie	Metryka	n	r	p
0,5B CoT (pilot)	cot_perplexity	150	-0,236	0,004
0,5B CoT (pilot)	cot_mean_token_entropy	150	-0,317	< 0,001
1,5B CoT	cot_perplexity	60	-0,283	0,028
1,5B CoT	cot_mean_token_entropy	60	-0,490	< 0,001
1,5B Agent (cała traj.)	agent_ppl_mean	60	-0,354	0,006
1,5B Agent (cała traj.)	agent_entropy_mean	60	-0,388	0,002
1,5B Agent (action)	action_ppl_mean	48	-0,436	0,002
1,5B Agent (action)	action_margin_mean	48	+0,454	0,001

Tabela 1: Korelacje Pearsona między metrykami a poprawnością w trzech ustawieniach (n to liczba obserwacji po usunięciu braków danych). Wartości policzone bezpośrednio z plików CSV w katalogu `overnight_results/`.

Z tabeli wyłaniają się dwa powtarzające się wzorce, niezależne od skali modelu i ustawienia. Po pierwsze, średnia entropia tokenowa i top-1 margin konsekwentnie wypadają lepiej niż samo perplexity jako predyktory poprawności - pełny rozkład prawdopodobieństw niesie więcej informacji niż prawdopodobieństwa tylko wybranych tokenów. Po drugie, wewnątrz agenta liczy

się to, na której części trajektorii mierzy się pewność: korelacje na segmentach **action** są silniejsze niż na całych trajektoriach, a te z kolei silniejsze niż na samym segmencie **answer**.

5 Kalibracja

Wyniki z sekcji 4 pokazują, że pewność modelu koreluje z poprawnością. Korelacja jest jednak własnością czysto porządkową - mówi tylko tyle, że pytania da się ustawić od najbardziej do najmniej ryzykownego. Nie odpowiada natomiast na pytanie, czy pewność rzędu 0,85 faktycznie oznacza 85% szans na poprawną odpowiedź. To już kwestia kalibracji i właśnie jej dotyczy ta sekcja.

Jako pewność modelu przyjęto $1/\text{PPL}$ - liczbę z przedziału $(0, 1]$. Jej zgodność z poprawnością mierzy Expected Calibration Error (ECE) na binach o równej liczbie obserwacji ($n_{\text{bins}} = 10$). Biny o równej liczności są tu potrzebne, bo przy dekodowaniu zachłannym wartości $1/\text{PPL}$ skupiają się blisko 1,0 (sekcja 3.4) i biny o stałej szerokości byłyby w większości puste.

Ustawienie	n	dokładność	ECE
1,5B CoT	60	60,0%	0,265
1,5B Agent (cała traj.)	60	28,3%	0,425
1,5B Agent (segmenty action)	48	25,0%	0,445

Tabela 2: Expected Calibration Error dla pewności zdefiniowanej jako $1/\text{PPL}$ (binowanie o równej liczności, $n_{\text{bins}} = 10$). Niższe ECE oznacza lepszą zgodność deklarowanej pewności z faktyczną poprawnością.

ECE jest wysokie w każdym ustawieniu (tabela 2). Diagramy niezawodności (plik `calibration_15b.png`) tłumaczą skąd to się bierze: w prawie każdym binie średnia pewność leży powyżej średniej poprawności, czyli model jest przesadnie pewny. W wariancie agentowym widać to najmocniej - w najniższych binach pewności model deklaruje 0,49-0,66, podczas gdy poprawność w tych binach wynosi 0%. Nawet tam, gdzie agent jest najmniej pewny, i tak jest pewniejszy, niż uzasadniałyby to jego wyniki.

Zestawienie tej obserwacji z sekcją 4 prowadzi do głównego wniosku. Perplexity liczone na rozumowaniu modelu istotnie koreluje z poprawnością, a mimo to daje wysokie ECE. Dobry porządek nie pociąga więc za sobą dobrej kalibracji - jedno z drugiego nie wynika.

Ma to konkretne znaczenie dla zastosowań. Tam, gdzie liczy się sama kolejność, perplexity się sprawdza: można nim wybrać najpewniejszą z kilku odpowiedzi albo wskazać najbardziej ryzykowne pytania do ręcznego sprawdzenia. Gorzej z regułami opartymi na stałym progu, w stylu „odroczy odpowiedź, gdy pewność spadnie poniżej 0,7” - wartość 0,7 nie oznacza tu 70% szans na poprawność, lecz znacznie mniej, więc taki próg trzeba najpierw

wykalibrować na danych z konkretnego zadania. Jest to zastrzeżenie wobec prac omawianych w sekcji 2.5: kierunek sygnału jest dobry, ale jego skala nie nadaje się do bezpośredniego prognozowania.

6 Dyskusja

Zebrane wyniki układają się w kilka spójnych wątków, które warto omówić razem - bo dopiero zestawione ze sobą i z literaturą z sekcji 2 mówią coś więcej niż pojedyncze współczynniki korelacji.

Pierwszy wątek dotyczy wyboru metryki. We wszystkich ustawieniach - od pilotażowego modelu 0,5B po agenta 1,5B - średnia entropia tokenowa i top-1 margin okazują się lepszymi predyktorami poprawności niż samo perplexity. Różnica jest niewielka, ale konsekwentna, i ma dość naturalne wytłumaczenie: entropia i margin korzystają z całego rozkładu prawdopodobieństw, a perplexity tylko z prawdopodobieństw tokenów, które model faktycznie wybrał. Model może wybrać token z wysokim prawdopodobieństwem, mając tuż za nim silną alternatywę - perplexity tego nie zauważy, a margin i entropia tak. Sugeruje to, że w dalszych pracach nad pewnością agentów warto sięgać po metryki rozkładowe, a nie ograniczać się do klasycznego PPL.

Drugi wątek to lokalizacja pomiaru wewnątrz agenta. Sygnał nie jest rozłożony równomiernie po trajektorii: najsilniej koreluje z poprawnością pewność na segmentach **Action**, słabiej na całej trajektorii, a na samym segmencie **Answer** znika niemal zupełnie. Wynika to z mechaniki ReAct - w momencie formułowania finalnej odpowiedzi wynik liczbowy jest już w kontekście, więc model go po prostu przepisuje i jest pewny niezależnie od tego, czy droga do tego wyniku była poprawna. Pewność niesie informację tam, gdzie model rzeczywiście podejmuje decyzję (co policzyć), a nie tam, gdzie tylko odtwarza gotowy rezultat. To ustalenie jest zbieżne z obserwacją Fanga i in. [3], że uśrednianie po całej długiej sekwencji rozcieńcza sygnał - tyle że tutaj rolę „nieistotnych” tokenów grają nie spójniki, lecz cała końcowa faza kopiowania odpowiedzi.

Trzeci, najważniejszy wątek dotyczy różnicy między korelacją a kalibracją (sekcja 5). Perplexity porządkuje pytania sensownie, ale jako bezwzględna miara pewności jest źle wykalibrowane - model jest systematycznie zbyt pewny siebie, i to tym bardziej, im trudniejsze ustawienie. Rozstrzyga to pozornie sprzeczne stanowiska z literatury. Gonen i in. [1] mają rację, że niższe PPL wiąże się z lepszymi wynikami - kierunek jest poprawny. Zastrzeżenia Wanga i in. [2] co do wiarygodności PPL też są jednak zasadne, jeśli traktować je jako uwagę o skali, a nie o porządku. Te dwa stanowiska nie są sprzeczne - opisują dwie różne własności tej samej liczby.

Płyne stąd praktyczny wniosek dla prac, które wbudowują pewność w pętlę decyzyjną agenta (sekcja 2.5). ReDAct [5] i pokrewne podejścia opie-

rają się na progowaniu surowej pewności, a wyniki tej pracy pokazują, że taki próg nie przenosi się między zadaniami bez rekaliibracji: ta sama wartość liczbowo oznacza inną faktyczną szansę poprawności w czystym CoT i w agencie. Sam pomysł sterowania niepewnością pozostaje sensowny, ale wymaga albo kalibracji na danych docelowych, albo oparcia decyzji na względnym uporządkowaniu, a nie na sztywnym progu.

Na koniec warto odnotować, że dwa główne wzorce - przewaga metryk rozkładowych oraz przeniesienie sygnału na segmenty **Action** - utrzymują się przy przejściu z modelu 0,5B na 1,5B oraz między ziarnami losowości. Przy skali tego projektu nie jest to dowód ogólności, ale sugeruje, że obserwacje nie są wyłącznie artefaktem jednego modelu czy jednego losowania danych.

7 Ograniczenia

Wyniki tej pracy trzeba czytać z kilkoma zastrzeżeniami.

Najpoważniejsze dotyczy dekodowania. Wszystkie eksperymenty korzystają z dekodowania zachłannego, przez co model na każdym kroku wybiera swój najbardziej prawdopodobny token, a wartości $1/\text{PPL}$ skupiają się blisko 1,0. Sygnał pewności jest więc ściśnięty i część informacji, którą niosłoby próbkowanie z pełnego rozkładu, zostaje utracona. Powtórzenie eksperymentów z włączonym samplowaniem mogłoby dać bogatszy obraz - i prawdopodobnie inne, być może niższe ECE.

Ograniczona jest też skala. Badano dwa małe modele z jednej rodziny (Qwen2.5 0,5B i 1,5B) na jednym zbiorze - GSM8K. To zadania arytmetyczne z dokładnym dopasowaniem odpowiedzi liczbowej, więc nie wiadomo, na ile obserwacje przenoszą się na większe modele ani na zadania o innym charakterze (otwarte odpowiedzi, kod, dłuższe trajektorie z wieloma narzędziami). Agent korzystał z jednego narzędzia - kalkulatora - co jest najprostszym możliwym ustawieniem ReAct.

Niewielkie są wreszcie próby. Korelacje liczone są na 60 (a w pilocie 150) obserwacjach, a w analizie segmentów **Action** efektywne n spada do 48. Przy takich liczebnościach przedziały ufności są szerokie - bootstrapowy 95% CI dla najmocniejszej korelacji sięga od $-0,60$ do $-0,24$ - więc raportowane współczynniki należy traktować jako rzędy wielkości i kierunki zależności, a nie jako precyzyjne oszacowania. Z tego samego powodu w pracy nie testowano rekaliibracji pewności; wskazano ją jedynie jako naturalny dalszy krok.

Bibliografia

- [1] Hila Gonen i in. "Demystifying Prompts in Language Models via Perplexity Estimation". W: *Findings of the Association for Computational*

- Linguistics: EMNLP 2023*. 2023. URL: <https://aclanthology.org/2023.findings-emnlp.679/>.
- [2] Yequan Wang i in. “Perplexity from PLM Is Unreliable for Evaluating Text Quality”. W: *arXiv preprint arXiv:2210.05892* (2022). URL: <https://arxiv.org/abs/2210.05892>.
 - [3] Lizhe Fang i in. “What Is Wrong with Perplexity for Long-Context Language Modeling?” W: *International Conference on Learning Representations (ICLR)*. 2025. URL: <https://arxiv.org/abs/2410.23771>.
 - [4] Mahdi Mohammadi i in. “Evaluation and Benchmarking of LLM Agents: A Survey”. W: *arXiv preprint arXiv:2507.21504* (2025). Accepted to KDD 2025. URL: <https://arxiv.org/abs/2507.21504>.
 - [5] Anonymous. “ReDAct: Uncertainty-Aware Deferral for LLM Agents”. W: *arXiv preprint arXiv:2604.07036* (2026). URL: <https://arxiv.org/abs/2604.07036>.
 - [6] Anonymous. “Every Response Counts: Quantifying Uncertainty of LLM-based Multi-Agent Systems through Tensor Decomposition”. W: *arXiv preprint arXiv:2604.08708* (2026). URL: <https://arxiv.org/abs/2604.08708>.
 - [7] Anonymous. “Towards Agents That Know When They Don’t Know: Uncertainty as a Control Signal for Structured Reasoning”. W: *arXiv preprint arXiv:2509.02401* (2025). URL: <https://arxiv.org/abs/2509.02401>.
 - [8] Karl Cobbe i in. “Training Verifiers to Solve Math Word Problems”. W: *arXiv preprint arXiv:2110.14168* (2021). URL: <https://arxiv.org/abs/2110.14168>.
 - [9] Jason Wei i in. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. W: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022.
 - [10] Shunyu Yao i in. “ReAct: Synergizing Reasoning and Acting in Language Models”. W: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.